Repo Tutor - Agent Design Doc

Job to be done

Help me own a code repository through a structured, evidence-backed review and a Socratic tutoring session that turns passive reading into active ownership. Per session, the agent produces: an elevator summary, a file-level responsibility map, entrypoint/data-flow tracing with evidence, top design decisions with tradeoffs, prioritized improvements, a graded Socratic script, and a confidence/UNCONFIRMED rating on every claim.

User & usage

Me (AI/ML engineer). One deep session per repo, 90–120 min, for repos ≤10k LOC (larger repos may need 1–2 extra sessions). Optional 15–30 min follow-ups.

Tools & data access plan

- **Repository**: Arena.ai GitHub connector, read-only, repo + branch selected at session start.
- **Uploaded docs**: architecture PDFs, design notes, README, tests, CI YAML. Allowed types: pdf, md, txt, csv, json. Never uploaded: .env, secrets, private keys.
- **Skills** (uploaded .md files; deterministic, reproducible tool wrappers): list_files, get_file_snippet, search_code, summarize_paths, grade_answer.

Setup: connect GitHub in Arena, select repo + branch, upload skill files and docs, load system prompt and guardrails into agent config. No external infra or paid connectors.

Draft instructions

System prompt: *You are Repo Tutor, a senior ML/systems architect and Socratic tutor. Use only the repository files and uploaded docs. For every factual claim attach evidence: file:line_range or uploaded_doc:page. If evidence is missing, mark UNCONFIRMED. Never run code, modify files, or reveal secrets without explicit APPROVE. Redact files matching (?i).secret.|.token.|.env.. Return confidence per claim (high/med/low).*
Workflow: list_files() → read README + top 10 files via get_file_snippet() / summarize_paths() → search_code() for entrypoint signals (**main**, Dockerfile CMD, Flask/FastAPI init) → build summary + component map → extract design decisions with snippets → prioritize improvements → run Socratic script, scoring via grade_answer() → halt for human approval before any execution or write action.

Eval cases (pre-defined, evidence required for each)

1. **Entrypoint detection:** returns correct startup file + snippet. Pass: correct file + snippet.
2. **Data flow**: traces ingest → preprocess → model → output with 2 file refs. Pass: all core steps + two refs.
3. **Decision extraction**: lists ≥3 design decisions with evidence. Pass: ≥2 correct with evidence.

4. **Test/CI coverage**: lists tests + CI config, recommends one reproducible test command. Pass: checks exist + command given.
5. **Teaching outcome**: after tutoring, I restate the system in ≤3 sentences. Rubric: Coverage 0–4, Correctness 0–4, Clarity 0–2 → pass if ≥7/10.

Each case's pass/fail and transcript are recorded for auditing.

Risks & guardrails

Risks: a malicious file (in-repo or uploaded) could attempt prompt injection; the redaction regex is pattern-based and can false-negative on unconventional secret formats; evidence citations can be technically present but selectively framed.
Must confirm (explicit APPROVE required): any code-executing command (pytest, pip install); any write/commit/push to the repo; any access beyond the connected repo and uploaded docs.
Must never do: modify/commit/push repo files; expose or print secrets (auto-redact via regex above); run arbitrary code without APPROVE.
Implemented: evidence requirement on every claim; confidence + UNCONFIRMED labeling; deterministic skill files (fixed I/O shape); human-in-loop gate on high-risk actions; regex secret detection.

Platform choice

Arena.ai (free), GitHub connector included, supports uploaded skill .md files, minimal setup, fast to demo.

- *vs. paid agent platforms (Claude/custom):* added cost and setup time not justified at this scope.
- *vs. custom LangChain pipeline:* infra, vector DB, and orchestration overhead exceed the 10-hour build target.

Model-agnostic by design: reproducibility rests on deterministic skills, evidence checks, and saved prompts; not on a specific underlying model.

Timebox (≈10h)

- Repo prep/docs (0.5h)
- Write skills (1.0h)
- Config + connectors (1.0h)
- Prompt + guardrails (2.0h)
- Indexing/retrieval testing (2.0h)
- Eval tuning (2.0h)
- Tutoring run + transcript (1.0h)

Acceptance checklist

☐ Saved agent (prompt + guardrails) in Arena
☐ Skills uploaded as .md
☐ GitHub connector configured
☐ Five eval cases documented and run
☐ Tutoring transcript saved
☐ This doc attached